API EBSI

Creado por Julian GONZALEZ AGUDELO, modificado por última vez por Alen Horvat el dic 02, 2019

Ticket	EBSIINT-169
Proposed	2019/10/17
Accepted	date
Driver	@Julian GONZALEZ AGUDELO
Approver	username
Consulted	@Andrew Rippon @Marc Antoine LEMAIRE
Informed	username

This document is open for comments until it gets accepted.

1. API EBSI

1.1. Decisions

The final decision will be taken by the DAG based on input coming from the Dev team and the architect team, on the following criteria:

- The relevance of the technical proposition (mature, open-source, well-supported product, proven track of record, modular approach, re-usability, etc.)
- Feasibility (within our time-frame)
- Compliance with our envision EBSI design platform and principle

1.2. Context

- For every implementation decision, we need to put on paper the rationals guiding our selections and decisions
- In order to be transparent, we are making this as an RFC (Request For Comment) allowing every expert to share their point of view, we will then review and take a final decision based on criteria described here above in the "Decisions" section

1.3. Consequences

- Based on decisions of the DAG, specific implementation will be selected

1.4. Alternatives Considered

Describe the considered alternatives (list and describe all the topics here under)

2. Available APIs

The EBSI API is a set of different APIs inside the node that provide services like off-chain storage and interaction with the blockchain. Below are the available APIs:

Panel > Blockchain Internal Home > EBSI Internal

> EBSIINT - Deliverables (WIP) > 1. Technical Group

API to interact directly with the blockchain, that is, to get balance, get transactions, get blocks, or even transfer or interact with a specific smart contract.

The list of all possible methods can be found here: https://github.com/ethereum/wiki/wiki/JSON-RPC

The entry point is https://api.ebsi.xyz/blockchain using POST.

Example to get block by number:

```
curl -X POST --data '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["0x1b4", true],"id":1}' https://api.ebsi.xyz/blockchai


{
  "id":1,
  "jsonrpc":"2.0",
  "result": {
    "number": "0x1b4",
    "hash": "0xe670ec64341771606e55d6b4ca35a1a6b75ee3d5145a99d05921026d1527331",
    "parentHash": "0x9646252be9520f6e71339a8df9c55e4d7619deeb018d2a3f2d21fc165dde5eb5",
    "nonce": "0xe04d296d2460cfb8472af2c5fd05b5a214109c25688d3704aed5484f9a7792f2",
    "sha3Uncles": "0x1dcc4de8dec75d7aab85b567b6ccd41ad312451b948a7413f0a142fd40d49347",
    "logsBloom": "0xe670ec64341771606e55d6b4ca35a1a6b75ee3d5145a99d05921026d1527331",
    "transactionsRoot": "0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cadc001622fb5e363b421",
    "stateRoot": "0xd5855eb08b3387c0af375e9cdb6acfc05eb8f519e419b874b6ff2ffda7ed1dff",
    "miner": "0x4e65fda2159562a496f9f3522f89122a3088497a",
    "difficulty": "0x027f07",
    "totalDifficulty": "0x027f07",
    "extraData": "0x0000000000000000000000000000000000000000000000000000000000000000",
    "size": "0x027f07",
    "gasLimit": "0x9f759",
    "gasUsed": "0x9f759",
    "timestamp": "0x54e34e8e"
    "transactions": [{...},{ ... }]
    "uncles": ["0x1606e5...", "0xd5145a9..."]
  }
}
```

3.1. Connection using authentication

There are some methods that are restricted to anonymous access, like send transaction. In this case the user must append a JWT authentication token in the headers of the query.

Example using authentication:

```
curl -X POST --data '{"jsonrpc":"2.0","method":"eth_syncing","params":[],"id":1}'
     -H "Authorization: Bearer xxxxxxxxxxxxxxxxx.yyyyyyyyyyyyyyyyyy.zzzzzzzzzzzzzzzzzzzzzzzzzzzzzz"
     https://api.ebsi.xyz/blockchain


{
  "id":1,
  "jsonrpc": "2.0",
  "result": false
}
```

4. Notary API

The notary API is used to read the current state of smart contract for notarization. The entry point for the API is https://api.ebsi.xyz/notary.

4.1. Get the contract address and ABI

To consult the contract address in Besu and the ABI to interact with it call https://api.ebsi.xyz/notary using GET

```
curl https://api.ebsi.xyz/notary

{
  "ok": true,
  "notary": {
    "address": "0x664D6EbAbbD5cf656eD07A509AFfBC81f9615741",
    "abi": [{ ... },{ ... }]
  }
}
```

4.2. Get record by hash

The records in the smart contract can be searched by the hash. Call https://api.ebsi.xyz/notary/:hash putting in hash the hash of the notarized document. If the hash exists in the smart contract it will return the timestamp (epoch time) and the address that performed the notarization.

Example:

```
curl https://api.ebsi.xyz/notary/0x23af7e059310debd807988d22320c4c50fd0099084e533e89c73f8e749ff1d7f

{
  "ok":true,
  "document": {
                                                                '3f8e749ff1d7f",
```

Alternatively, the smart contract can be read by the normal calls using web3.js.

Example:

```
var timestamp = await contract.methods.timestamp(hash).call()
```

4.3. How to notarize a hash

The Notary API is used for read-only. Use the blockchain besu API in order to send transactions.

Code example using web3.js library:

```
var axios = require('axios')
var Web3 = require('web3')

var token = 'xxxxxxxxxxxxxxxxxxxxxxx.yyyyyyyyyyyyyyyyyyyyyyyyy.zzzzzzzzzzzzzzzzzzzzz' // use login to authenticate an
var privKey = '0x81e4b01ba124f35f521fc83ff6c11bdbf8d31b21a1765f165af09dd965fc832f'
var hash = '0x123456789'
var rpc_node = 'https://api.ebsi.xyz/blockchain'

(async ()=>{
  // get contract address and abi
  var response = await axios.get('https://api.ebsi.xyz/notary')
  var notary = response.data.notary

  var web3 = new Web3(new Web3.providers.HttpProvider(rpc_node))
  var from = web3.eth.accounts.privateKeyToAccount(privKey).address
  var contract = new web3.eth.Contract(notary.abi, notary.address)

  var data = contract.methods.addRecord( hash ).encodeABI()

  var txJSON = {
    gasPrice: web3.utils.numberToHex(0),
    gasLimit: web3.utils.numberToHex(221000),
    to: notary.address,
    value: web3.utils.numberToHex(web3.utils.toWei('0', 'ether')),
    nonce: await web3.eth.getTransactionCount(from, 'pending'),
    data: data
  }

  var signed = await web3.eth.accounts.signTransaction(txJSON, privKey)
  var query = {jsonrpc:'2.0', method: 'eth_sendRawTransaction', params: [signed.rawTransaction], id: 1}
  var opts = { headers: {"Authorization" : `Bearer ${token}`} }
  var response = await axios.post(rpc_node, query, opts)
  console.log(response.data)
})()
```

5. Storage API

The storage API is used to store documents off-chain but adding the possibility to be replicated between the member states. The are 3 databases to store documents:

- **Gluster FS:** The data is replicated and distributed between the different nodes.
- **Cassandra:** The data is replicated and distributed between the different nodes.
- **Mongo:** The data is not replicated, only stored in the node.

This API requires authentication: JWT token in the headers and the role "file-storage".

5.1. Store documents

The entry point to store files is https://api.ebsi.xyz/file-storage/store using POST. The data must be a multi-part request containing the following fields:

- File
- "database": Select the database to store the file. It could be "mongo", "cassandra", or "gluster-fs".

Example:

```

var fs = require('fs')
var FormData = require('form-data')
var axios = require('axios')

var filename = './my_file.pdf'
var database = 'cassandra' // user 'mongo', 'cassandra', or 'gluster-fs'

const fileData = fs.readFileSync(filename)
var form = new FormData();
form.append('my_file', fileData, filename);
form.append('database', database)
```

```
var opts = {headers:{post:form.getHeaders()}}
if(token)
  opts.headers.Authorization = `Bearer ${token}`
var response = await axios.post('https://api.ebsi.xyz/file-storage/store', form, opts)
console.log(response.data)
```

The API will return an acknowledgement message including the hash of the file:

```
{
  ok: true,
  message: 'File stored',
  hash: '0xd82f1272b541c207c09a62302734eb5e0519aea3489ff93a96173ca485a342e3'
}
```

5.2. Get documents

The entry point to get documents is https://api.ebsi.xyz/file-storage/:hash, where :hash is the identifier of the file. The api will search the hash in gluster-fs, cassandra, and mongo (in this order).

This call requires authentication JWT token in the headers of the query.

Example:

https://api.ebsi.xyz/file-storage/0x2765899c7a8a5cd76d175858d502c77c627020ae82419c9b442d11ea9c9148b1

```
curl
  -H "Authorization: Bearer xxxxxxxxxxxxxxxxx.yyyyyyyyyyyyyyyyyy.zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz"
  https://api.ebsi.xyz/file-storage/0x2765899c7a8a5cd76d175858d502c77c627020ae82419c9b442d11ea9c9148b1

# Demo file

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore
magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.
Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
```

5.2.1. Get documents from a specific database

Using the call https://api.ebsi.xyz/file-storage/:hash/:database the file is searched only in the specified database.

Example to get a file from cassandra:

https://api.ebsi.xyz/file-storage/0x2765899c7a8a5cd76d175858d502c77c627020ae82419c9b442d11ea9c9148b1/cassandra

```
curl
  -H "Authorization: Bearer xxxxxxxxxxxxxxxxx.yyyyyyyyyyyyyyyyyy.zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz"
  https://api.ebsi.xyz/file-storage/0x2765899c7a8a5cd76d175858d502c77c627020ae82419c9b442d11ea9c9148b1/cassandra

# Demo file

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore
magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.
Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
```

6. Wallet requests storage API

This API is used by the Interaction Manager of the wallet in order to store the queue of messages to be processed by the wallet (messages coming from the Relying parties). This information is stored in Cassandra in order to be replicated between the member states. This API requires authentication and the role "wallet", which means that it is not the end user who authenticates but the wallet application. The entry point for the API is https://api.ebsi.xyz/wallet-request-storage.

6.1. Insert request

To insert a new request in the queue call https://api.ebsi.xyz/wallet-request-storage/insert using POST. The post data must contain "emisor" (public key), "receptor" (public key), and "message" (free JSON structure).

Example to insert a request

```
curl -X POST
  --data '{
    "emisor":"0xCB26f311E0b669D1a396967768A7f23532FD1010",
    "receptor":"0xBD8D51B4D3fE76Ae746342eCC7E50E7e5bAc78a0",
    "message":{"date":"2019-11-11T10:05:57"} }'
  -H "Authorization: Bearer xxxxxxxxxxxxxxxxx.yyyyyyyyyyyyyyyyyyyyy.zzzzzzzzzzzzzzzzzzz"
  https://api.ebsi.xyz/wallet-request-storage/insert

{
  "ok":true,
  "message":"Message inserted"
}
```

6.2. Update request

To update a request call https://api.ebsi.xyz/wallet-request-storage/update using POST. The post data must contain the "id" of the request, "emisor" (public key), "receptor" (public key), and "message" (free JSON structure).

```
    "receptor":"0xBD8D51B4D3fE76Ae746342eCC7E50E7e5bAc78a0",
    "message":{"date":"2019-11-12T10:05:57"} }'
  -H "Authorization: Bearer xxxxxxxxxxxxxxxxx.yyyyyyyyyyyyyyyyyyyyy.zzzzzzzzzzzzzzzzzzz"
  https://api.ebsi.xyz/wallet-request-storage/update

{
  "ok":true,
  "message":"Message updated"
}
```

6.3. Delete request

To delete a request call https://api.ebsi.xyz/wallet-request-storage/delete using POST. The post data must contain the "id" of the request.

Example to delete a request

```
curl -X POST
  --data '{"id":"5fbb9f10-0242-11ea-93ec-9116fc548b6b"}'
  -H "Authorization: Bearer xxxxxxxxxxxxxxxxx.yyyyyyyyyyyyyyyyyyyyy.zzzzzzzzzzzzzzzzzzz"
  https://api.ebsi.xyz/wallet-request-storage/delete

{
  "ok":true,
  "message":"Message deleted"
}
```

6.4. Get receptor queue

To get the receptor queue call https://api.ebsi.xyz/wallet-request-storage/:receptor using GET, and putting in ":receptor" the public key of the receptor.

Example to get the receptor queue

```
curl -H "Authorization: Bearer xxxxxxxxxxxxxxxxx.yyyyyyyyyyyyyyyyyyyyy.zzzzzzzzzzzzzzzzzzz"
  https://api.ebsi.xyz/wallet-request-storage/0xBD8D51B4D3fE76Ae746342eCC7E50E7e5bAc78a0

{
  "ok": true,
  "queue": [{
    "id": "c15e2830-023f-11ea-93ec-9116fc548b6b",
    "emisor": "0xCB26f311E0b669D1a396967768A7f23532FD1010",
    "message": "{\"date\":\"2019-11-08T15:52:23\"}",
    "receptor": "0xBD8D51B4D3fE76Ae746342eCC7E50E7e5bAc78a0",
    "timestamp": "2019-11-08T15:52:23.859Z"
  },
  {
    "id": "2453e870-0241-11ea-a4db-4b7a475a76be",
    "emisor": "0xCB26f311E0b669D1a396967768A7f23532FD1010",
    "message": "{\"date\":\"2019-11-08T16:02:19\"}",
    "receptor": "0xBD8D51B4D3fE76Ae746342eCC7E50E7e5bAc78a0",
    "timestamp": "2019-11-08T16:02:19.383Z"
  },
  {
    "id": "73a2af00-0242-11ea-93ec-9116fc548b6b",
    "emisor": "0xCB26f311E0b669D1a396967768A7f23532FD1010",
    "message": "{\"date\":\"2019-11-08T16:11:41\"}",
    "receptor": "0xBD8D51B4D3fE76Ae746342eCC7E50E7e5bAc78a0",
    "timestamp": "2019-11-08T16:11:41.936Z"
  }]
}
```

6.5. Get emisor queue

To get the emisor queue call https://api.ebsi.xyz/wallet-request-storage/emisor/:emisor using GET, and putting in ":emisor" the public key of the emisor.

Example to get the receptor queue

```
curl -H "Authorization: Bearer xxxxxxxxxxxxxxxxx.yyyyyyyyyyyyyyyyyyyyy.zzzzzzzzzzzzzzzzzzz"
  https://api.ebsi.xyz/wallet-request-storage/emisor/0xCB26f311E0b669D1a396967768A7f23532FD1010

{
  "ok": true,
  "queue": [{
    "id": "c15e2830-023f-11ea-93ec-9116fc548b6b",
    "emisor": "0xCB26f311E0b669D1a396967768A7f23532FD1010",
    "message": "{\"date\":\"2019-11-08T15:52:23\"}",
    "receptor": "0xBD8D51B4D3fE76Ae746342eCC7E50E7e5bAc78a0",
    "timestamp": "2019-11-08T15:52:23.859Z"
  },
  {
    "id": "2453e870-0241-11ea-a4db-4b7a475a76be",
    "emisor": "0xCB26f311E0b669D1a396967768A7f23532FD1010",
    "message": "{\"date\":\"2019-11-08T16:02:19\"}",
    "receptor": "0xBD8D51B4D3fE76Ae746342eCC7E50E7e5bAc78a0",
    "timestamp": "2019-11-08T16:02:19.383Z"
  },
  {
    "id": "73a2af00-0242-11ea-93ec-9116fc548b6b",
```

7. Login

The login module is used to authenticate a user before using a restricted service provided by the node. Currently, the unique restricted service is the interaction with the blockchain, only entities will be able to send transactions.

The entry point to login is https://api.ebsi.xyz/login using POST. There are two ways to login:

1. Using username & password
2. Using public key + signature.

7.1.1. Login with username & password

In the POST data include username and password in json format. If the authentication is OK the API will return a JWT token.

```
curl -X POST --data '{"username":"admin","password":"admin"}' https://api.ebsi.xyz/login


{
  ok: true,
  token: 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwiZ3JvdXBzIjpbXSwic
          m9sZXMiOlsiYWRtaW4iXSwiaWF0IjoxNTcyOTY0OTc0LCJleHAiOjE1NzMwNTEzNzR9.7rIBU1JhfrTbWu
          Dml5LVE-4iutcxISSb2PhX569bYAQ'
}
```

7.1.2. Login with public key & signature

The user needs to login signing a message with his private key. The structure of the message is as follows:

```
message = {
  date:       "2019-10-17T10:00:00",
  expiration: "2019-10-17T10:05:00",
}
```

The "date" is the actual date, and the "expiration" the expiration time of the signature. It cannot be longer than 10 minutes. This structure prevents replay attacks.

The signature of this message can be done using web3.js:

```
var Web3 = require('web3')
var message = {date:"2019-10-17T10:00:00",expiration:"2019-10-17T10:05:00"}
var private_key = '0x81e4b01ba124f35f521fc83ff6c11bdbf8d31b21a1765f165af09dd965fc832f'
var signature = await new Web3().eth.accounts.sign(JSON.stringify(message), private_key)
```

This signature is the POST data. This is how it looks:

```
{
  message: '{"date":"2019-10-17T10:00:00","expiration":"2019-10-17T10:05:00"}',
  messageHash: '0x1da44b586eb0729ff70a73c326926f6ed5a25f5b056e7f47fbc6e58d86871655',
  v: '0x1c',
  r: '0xb91467e570a6466aa9e9876cbcd013baba02900b8979d43fe208a4a4f339f5fd',
  s: '0x6007e74cd82e037b800186422fc2da167c747ef045e5d18a5f5d4300f8e1a029',
  signature: '0xb91467e570a6466aa9e9876cbcd013baba02900b8979d43fe208a4a4f339f5fd6007e74cd82e037b800186422fc2da167c747ef045e5d18a5f5d
}
```

If the authentication is OK the API will return a token:

```
{
  ok: true,
  token: 'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx.yyyyyyyyyyyyyyyyyyyyyyyyyyyyyy.zzzzzzzzzzzzzzz'
}
```

8. Resources

- Web3js - https://web3js.readthedocs.io/en/v1.2.0/index.html
- JSON RPC ethereum - https://github.com/ethereum/wiki/wiki/JSON-RPC